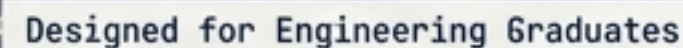


A Technical Deep Dive into Transformer Architectures and Agentic Workflows



The Atomic Unit: Tokens & High-Dimensional Space

From Text to Vector

smart ##er

Token IDs

[4521, 102]

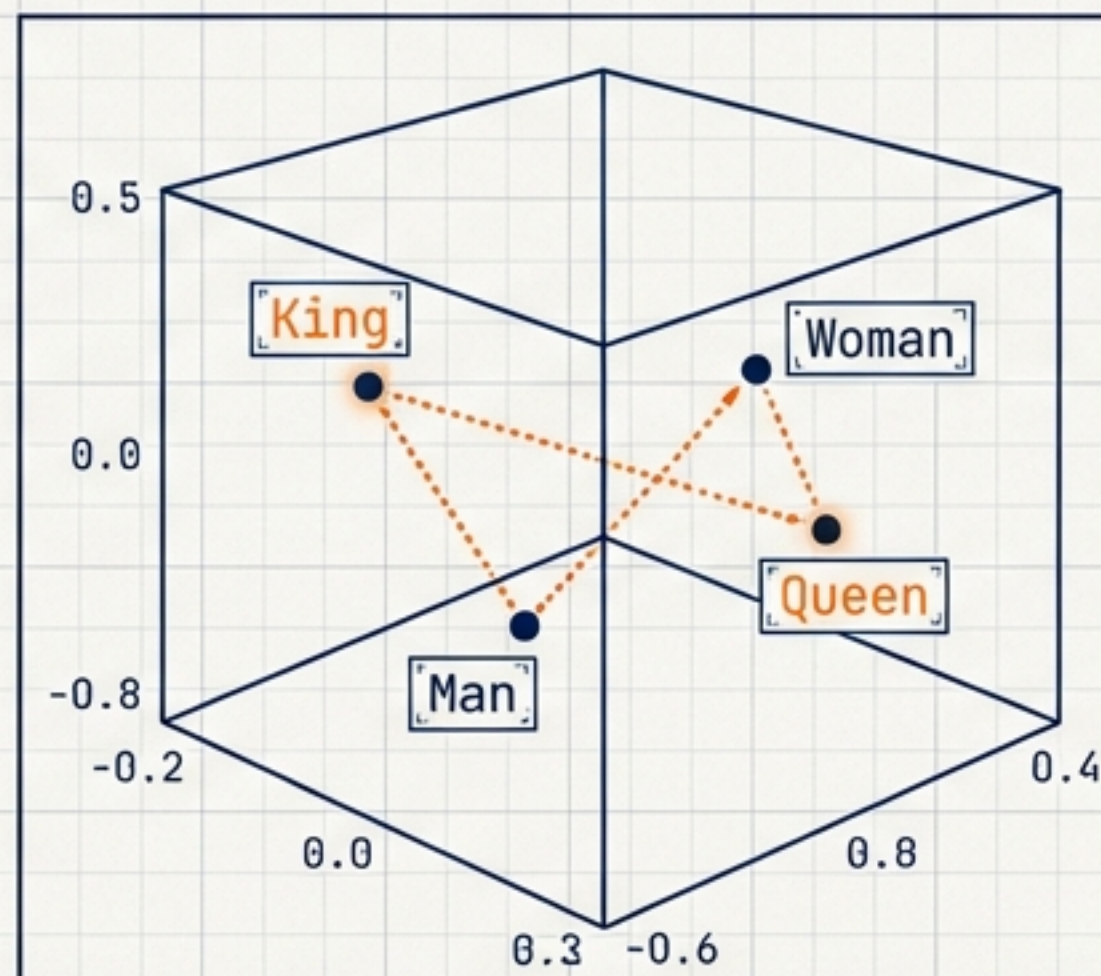
Dense Vector ($d_{\text{model}}=768$)

0.021
-0.553
1.201
0.876
-0.004
...
0.349
-1.115
...

Tokenization: Decomposing text into subword units to optimise vocabulary efficiency (approx. 50k tokens representing millions of words).

Embeddings: A learnable lookup table converting integer indices into dense, high-dimensional vectors.

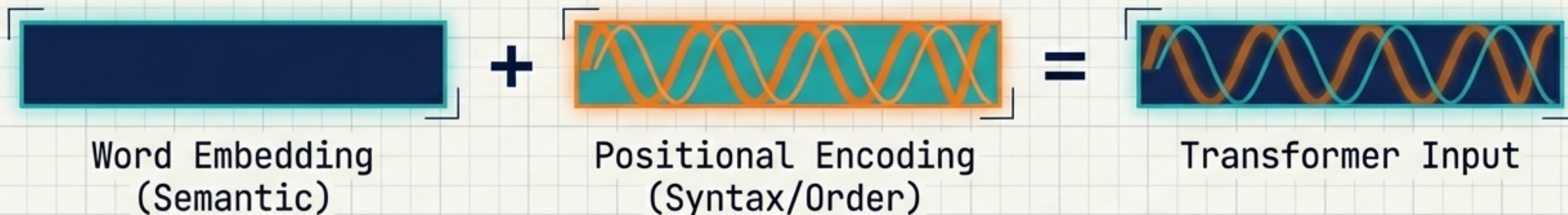
Vector Space: Semantic meaning is encoded as geometric proximity.



$$\text{Vector}(\text{King}) - \text{Vector}(\text{Man}) + \text{Vector}(\text{Woman}) \approx \text{Vector}(\text{Queen})$$

Preserving Order: Positional Encodings

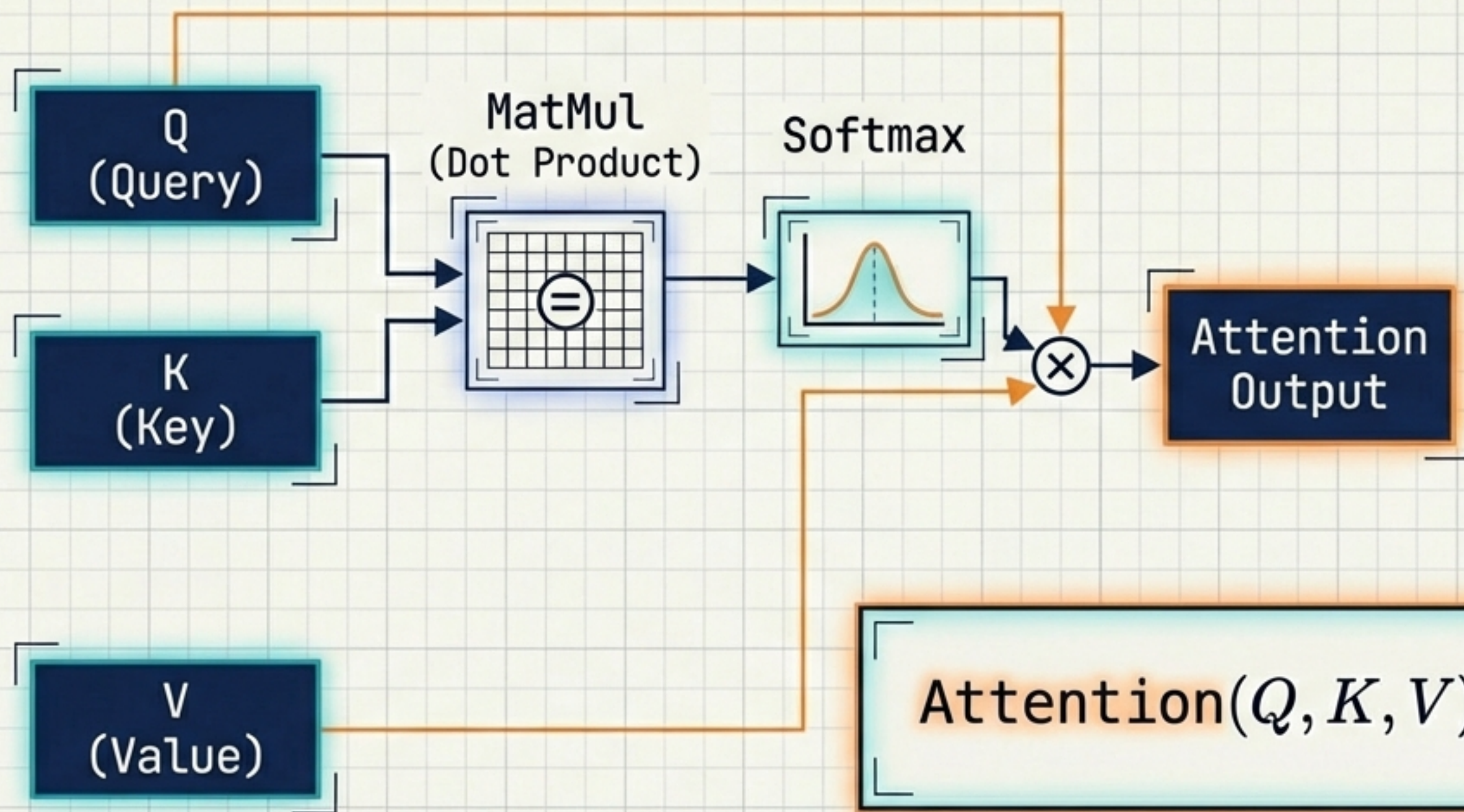
Vector Addition: Word Embedding + Positional Signal



$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

- **The Problem:** Transformers process tokens in parallel, lacking inherent sequential awareness (unlike RNNs).
- **The Solution:** Injecting geometric signals into the embedding to represent relative positions.
- **Methodology:** Sinusoidal frequencies or Rotary Positional Embeddings (RoPE) encode distance as rotation.

The Core Mechanism: Scaled Dot-Product Attention



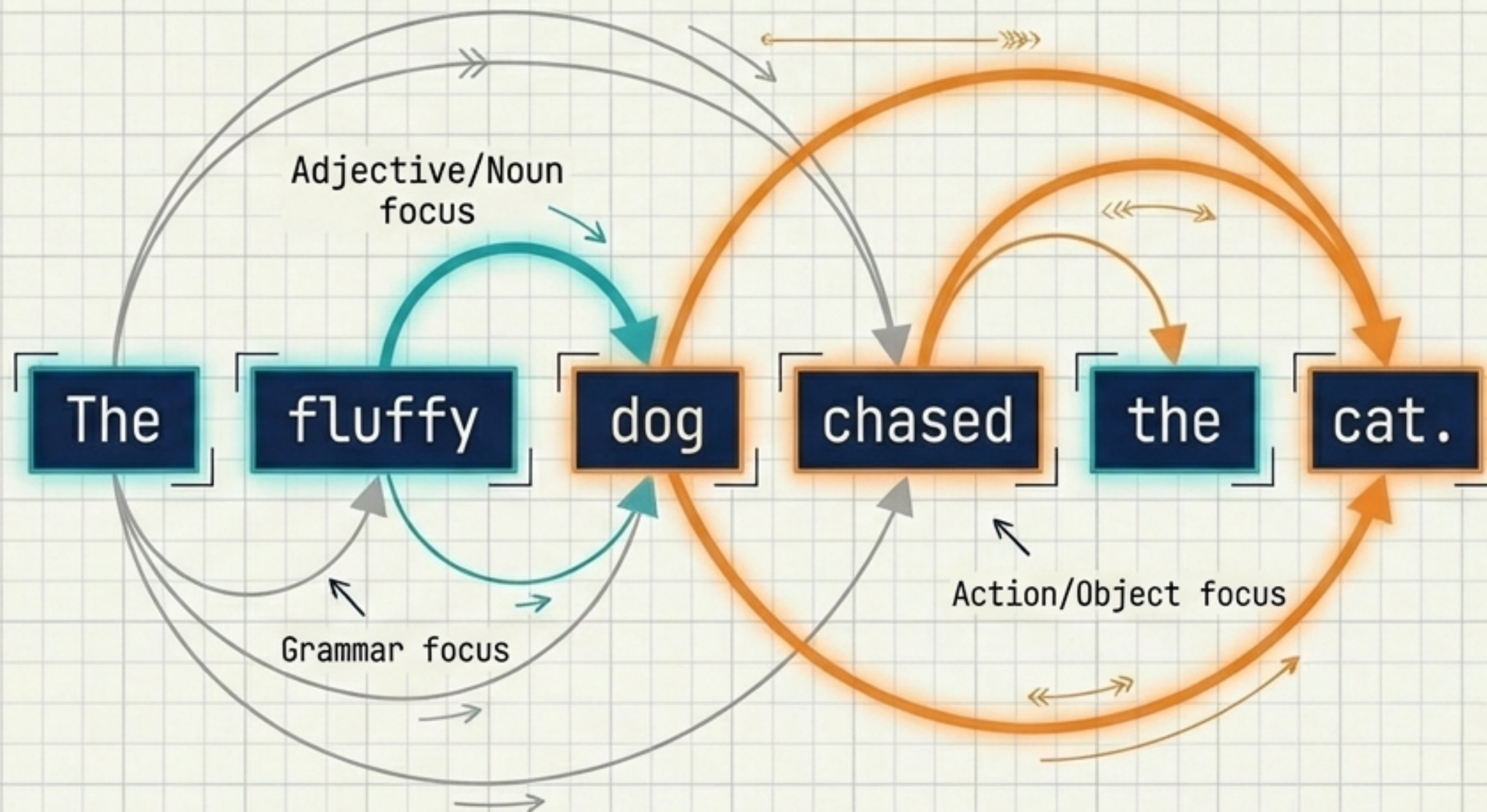
The Retrieval Analogy

- **Query:** What am I looking for?
- **Key:** What defines me?
- **Value:** What information do I hold?

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

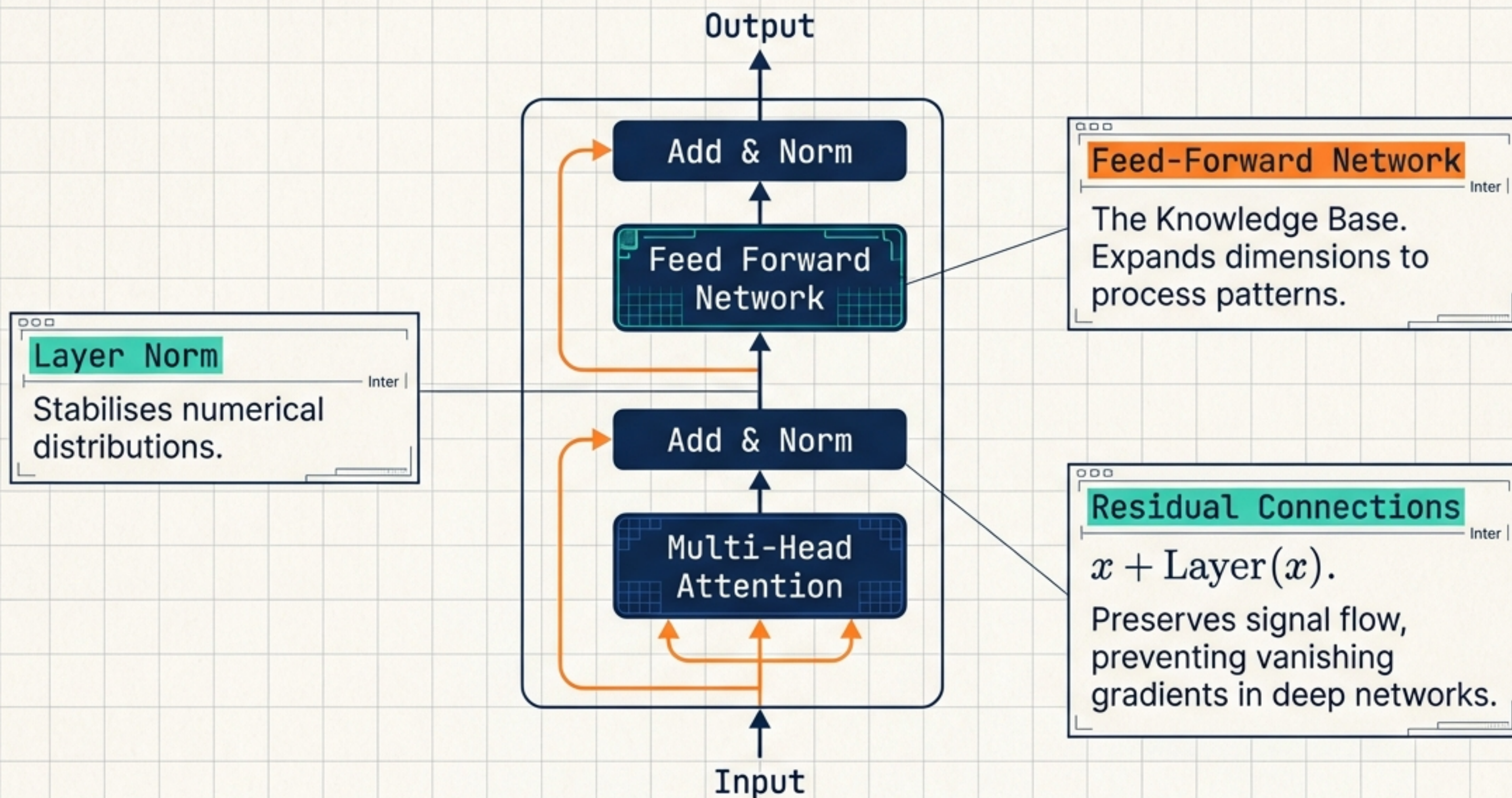
The model calculates **relevance scores** between tokens to create a **weighted sum** of **values**, prioritising contextually relevant information.

Parallel Perspectives: Multi-Head Attention



- **Concept:** The embedding dimension is split into multiple parallel 'heads' (e.g., 8 to 128).
- **Specialisation:** Each head independently learns to track different linguistic features (syntactic, semantic, tonal).
- **Synthesis:** Outputs are concatenated and projected back to the model dimension.

The Transformer Block: FFNs & Residuals



The Interface: Prompting Strategies

```
messages = [  
    # System Role: Instructions  
    {"role": "system", "content": "You are a highly accurate  
sentiment classifier."},  
  
    # Few-Shot Examples (The "Shots")  
    {"role": "user", "content": "Text: 'I love this!'"},  
    {"role": "assistant", "content": "Positive"},  
    {"role": "user", "content": "Text: 'This is terrible.'"},  
    {"role": "assistant", "content": "Negative"},  
  
    # Zero-Shot / Actual Query  
    {"role": "user", "content": "Text: 'I am absolutely happy.'"}  
]
```

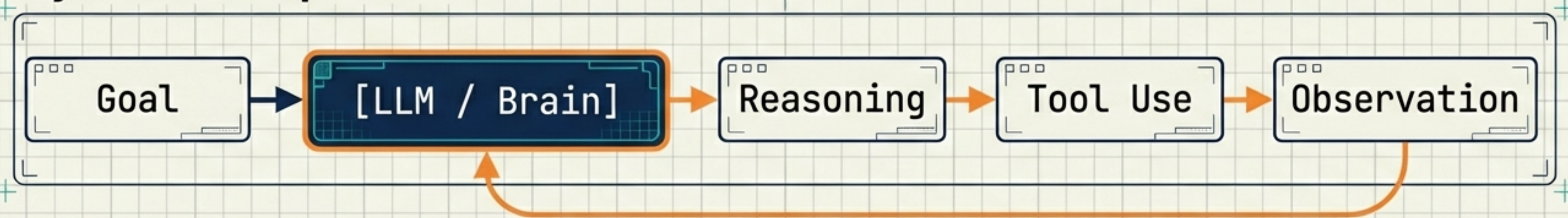
- **In-Context Learning:** Steering the model without updating weights.
- **Zero-Shot:** Relying on pre-trained knowledge.
- **Few-Shot:** Providing examples in the context window to guide pattern matching and format adherence.

From Passive Models to Agentic AI

Stateless & Isolated

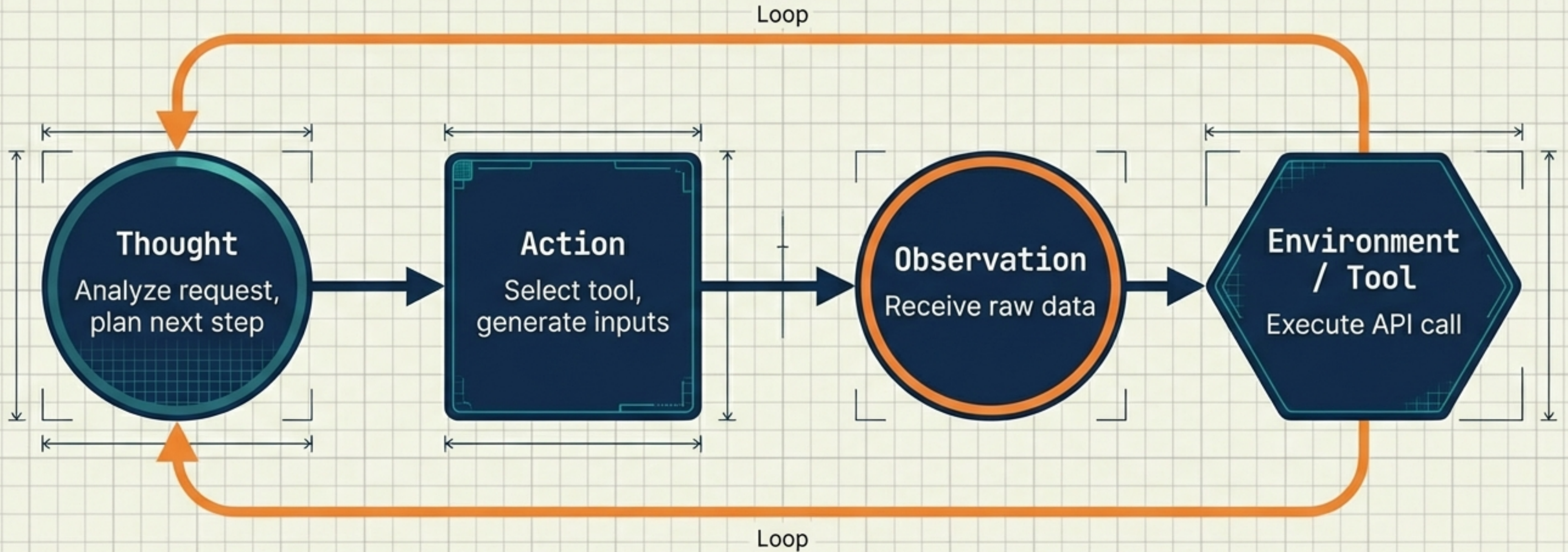


Dynamic & Loop-Based



- **The Controller:** The LLM shifts from being just a writer to an orchestrator.
- **Capabilities:** Reasoning, Planning, Tool Execution, and Memory.

The ReAct Pattern (Reason + Act)



Cyclic dependency mimicking human problem-solving loops.

Implementing Agency: OpenAI SDK

```
from openai import tool, AssistantAgent
```

```
# 1. Define the Tool
```

```
@tool
```

```
def get_weather(city: str) -> str:
```

```
    """Returns current weather for a specific city."""
```

```
    return f"The weather in {city} is 29°C."
```

```
# 2. Initialize the Agent
```

```
agent = AssistantAgent(
```

```
    tools=[get_weather],
```

```
    model="gpt-4o",
```

```
    instructions="You are a helpful assistant."
```

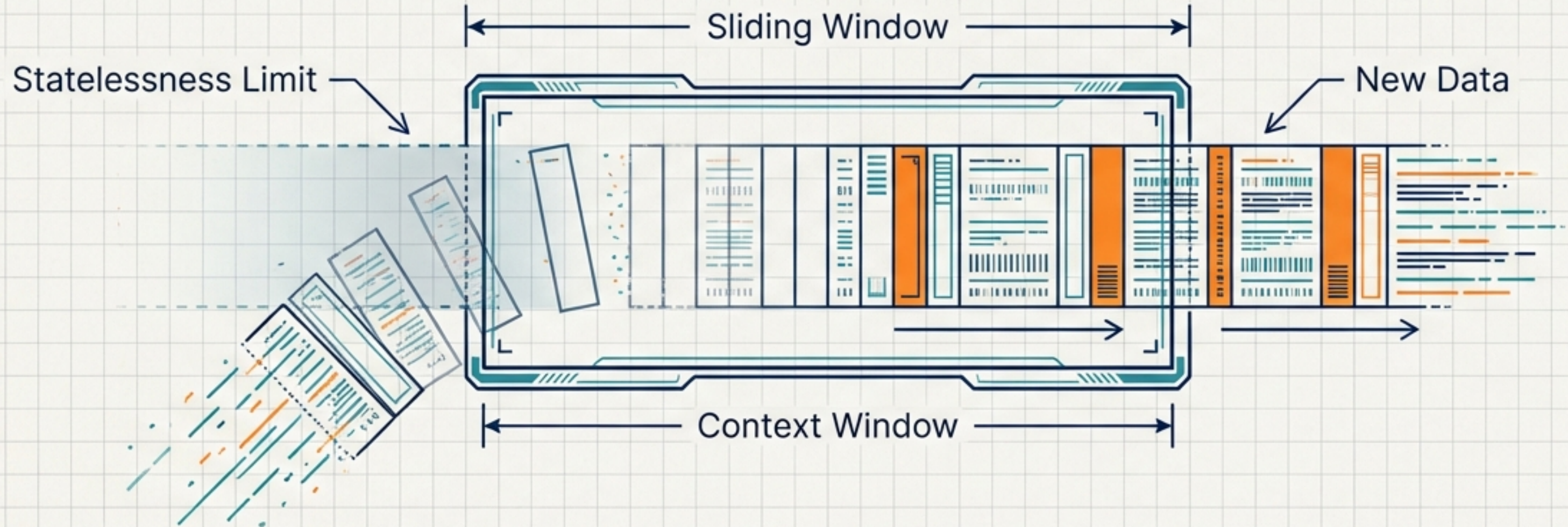
```
)
```

```
# The agent autonomously determines when to call get_weather
```

Schema
Generation

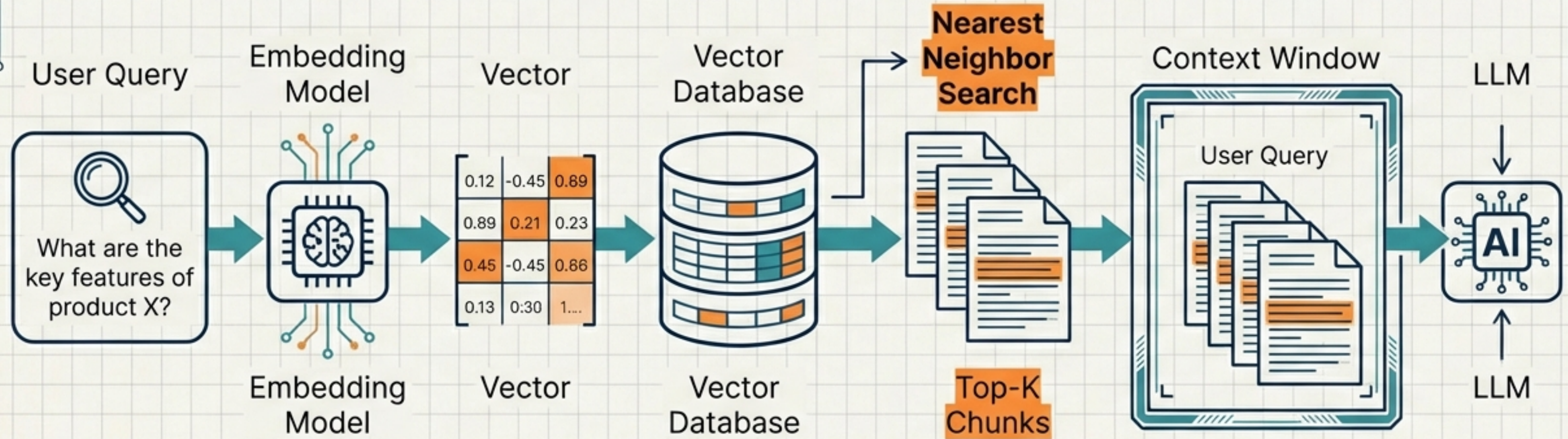
Tool
Injection

The "Goldfish Problem": Context & Memory



- **Statelessness**: LLMs treat every API call as a fresh start. No inherent persistence.
- **Short-Term Memory**: Limited by token costs and compute window (e.g., 128k tokens).
- **The Challenge**: How to retain knowledge from a 10,000-page manual or past interactions?

Long-Term Memory: RAG & Vector Databases



- The "Open Book" exam approach. **RAG (Retrieval-Augmented Generation).**
- **Mechanism:** Semantic search retrieves relevant knowledge chunks to inject into the prompt context dynamically.

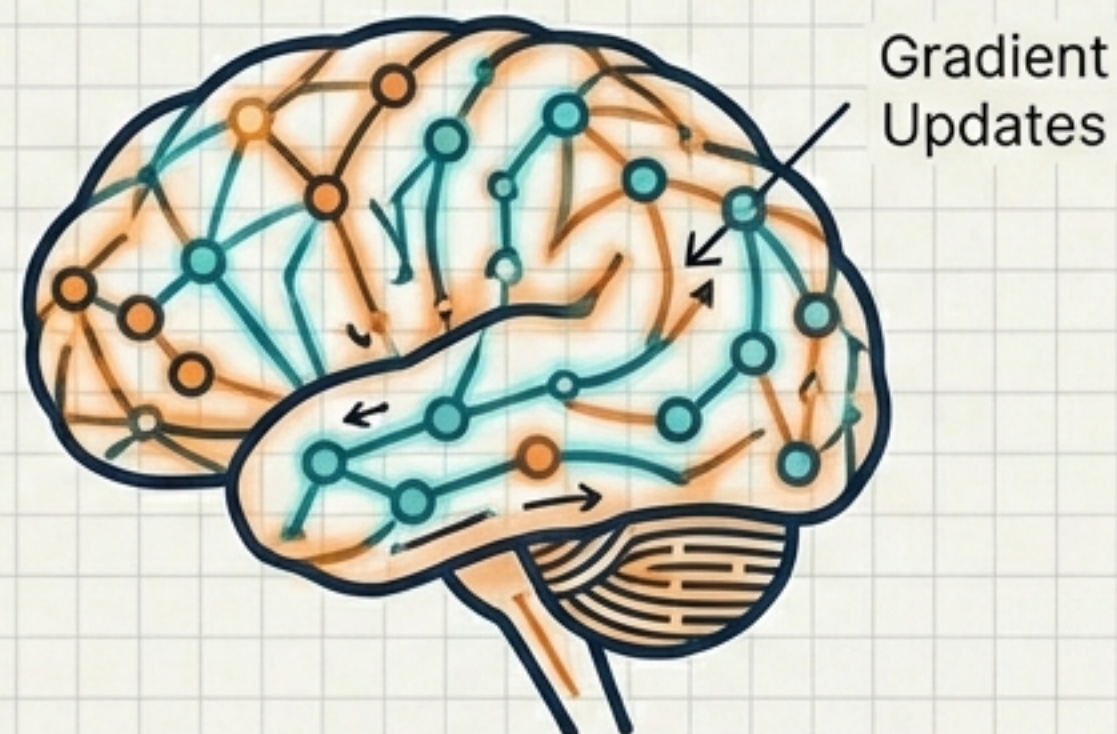
Fine-Tuning: Customizing the Engine

PROMPTING



Temporary Context Injection

FINE-TUNING



Permanent Parameter Updates

- **Prompting:** Optimises the input. Good for general knowledge.
- **Fine-Tuning:** Optimises the weights. Crucial for modifying behavior, tone, and strict format adherence.

The Impact of Fine-Tuning

Input: "Extract status."

Generic Model (Bad)

Sure! Based on the text you provided, it seems like the system is running okay, but there was a small error log...

Verbose & Unstructured

Fine-Tuned Model (Good)

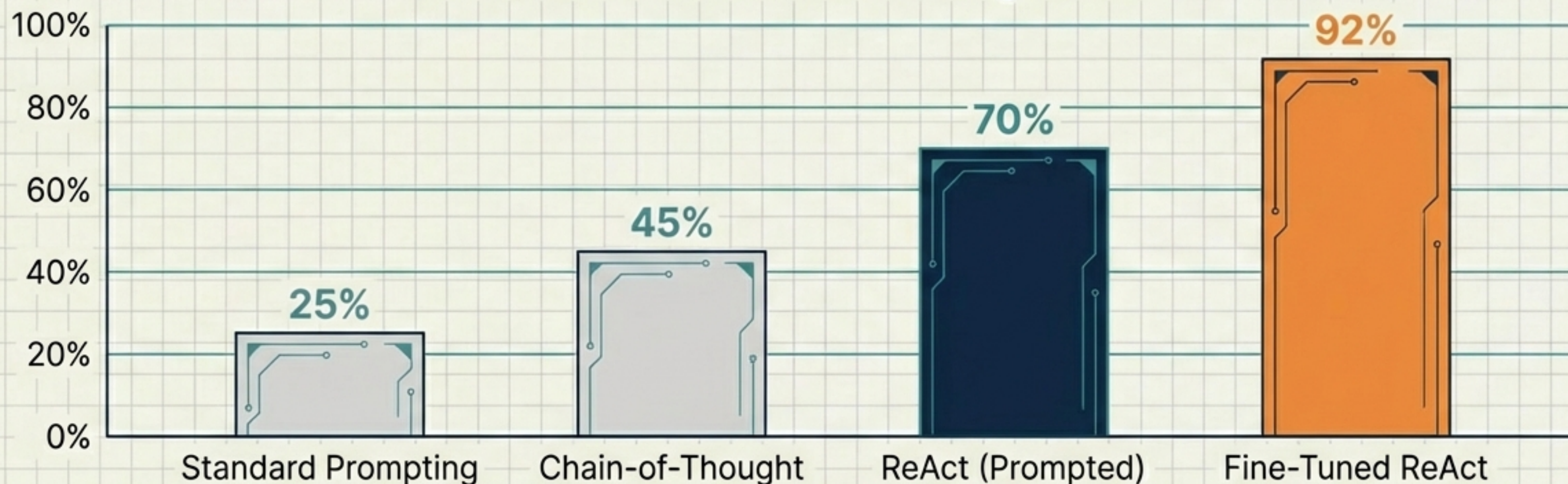
```
{  
  "status": "degraded",  
  "error_code": 404,  
  "latency_ms": 120  
}
```

Schema-Compliant JSON

Fine-tuning enforces strict output schemas for reliable engineering integration.

Performance Comparison: Methods of Agency

Success Rate on Reasoning Tasks



Source: ReAct / Cognitive Architecture Research.

Key takeaway: The synergy of architectural structure **(ReAct)** and **behavioral training (Fine-Tuning)** yields the most robust agents.

The Future: Cognitive Architectures (CoALA)



- **Shift in Paradigm:** Moving from simple Chatbots to Cognitive Architectures for Language Agents **(CoALA)**.
- **Conclusion:** The future of AI engineering is designing systems *around models* to enable true autonomy.